

Operating Systems

Assessment Test 1 (AT1) | Course Outcome 4 (CO4)
File Allocation Strategies & Disk Scheduling Algorithms

Question 1: Simulation of File Allocation Methods (Contiguous, Linked, Indexed)

[5 Marks]

Question as given: Write a program to simulate the three file allocation methods—Contiguous, Linked, and Indexed. The program should accept inputs including the total disk size (number of blocks), the file size (number of blocks required), and the allocation method (1 for Contiguous, 2 for Linked, 3 for Indexed). Based on the selected method, the program must allocate disk blocks accordingly, display the allocated block numbers, and handle fragmentation scenarios, particularly when contiguous allocation fails due to insufficient continuous space. The output should show the list of allocated blocks or an appropriate error message if allocation is not possible. Ensure that built-in memory allocation functions are not used, and assume that disk blocks are numbered starting from 0.

What This Question Is Asking

- The question is asking for one program that can imitate all three ways an OS can give disk blocks to a file, selected using a menu choice (1/2/3).
- “Handle fragmentation” means the program must actually detect the case where enough free blocks exist in total, but not as one continuous run, and print a clear error instead of allocating.
- “No built-in memory allocation functions” means malloc(), calloc(), realloc() cannot be used — fixed-size arrays represent the disk and every file's blocks instead.
- Disk blocks are numbered 0 to (total_blocks - 1), and the free/allocated state of the disk is shown before and after the operation.

Program Logic

- Represent the disk as an array disk[MAX] where 0 = free, 1 = allocated.
- Contiguous: scan for the first index i such that blocks $i..i+fileSize-1$ are all free; if no such run exists, report fragmentation failure.
- Linked: scan the whole disk left to right and pick any fileSize free blocks (need not be adjacent); print them as a chain block -> block -> ... -> NULL.
- Indexed: reserve one extra free block as the index block, then pick fileSize more free blocks as data blocks; print the index block and the data block list it stores.

C Program

```

/* =====
Q1: Simulation of File Allocation Methods
- Contiguous Allocation
- Linked Allocation
- Indexed Allocation
No built-in dynamic memory functions (malloc/calloc/realloc) are used.
All storage is implemented using static (fixed size) arrays.
===== */

#include <stdio.h>

#define MAX 50

int disk[MAX];          /* 0 = free block, 1 = allocated block */
int total_blocks;

/* ----- helper functions ----- */

void displayDiskStatus()
{
    int i;
    printf("\nCurrent Disk Block Status (0-Free, 1-Allocated):\n");
    for (i = 0; i < total_blocks; i++)
        printf("Block %d : %d\n", i, disk[i]);
}

int countFreeBlocks()
{
    int i, count = 0;
    for (i = 0; i < total_blocks; i++)
        if (disk[i] == 0)
            count++;
    return count;
}

/* ----- Contiguous Allocation ----- */

void contiguousAllocation(int fileSize)
{
    int i, j, start = -1, found;

    for (i = 0; i <= total_blocks - fileSize; i++)
    {
        found = 1;
        for (j = i; j < i + fileSize; j++)
        {
            if (disk[j] == 1)
            {
                found = 0;
                break;
            }
        }
        if (found)
        {
            start = i;
            break;
        }
    }

    if (start == -1)
    {
        printf("\nCONTIGUOUS ALLOCATION FAILED!\n");
    }
}

```

```

        printf("Reason : No set of %d CONTIGUOUS free blocks is available.\n", fileSize); printf("This is
a classic case of EXTERNAL FRAGMENTATION -- free\n"); printf("blocks exist on the disk, but
they are scattered and not\n"); printf("adjacent to each other, so a contiguous run cannot be
formed.\n");
        return;
    }

    for (j = start; j < start + fileSize; j++)
        disk[j] = 1;

    printf("\nContiguous Allocation Successful!\n");
    printf("File occupies blocks %d to %d\n", start, start + fileSize - 1);
    printf("Allocated Block List : ");
    for (j = start; j < start + fileSize; j++)
    {
        printf("%d ", j);
    }
    printf("\n");
/* -----2. Linked Allocation ----- */

void linkedAllocation(int fileSize)
{

    int allocated[MAX];
    int count = 0;
    int i;

    if (countFreeBlocks() < fileSize)
    {
        printf("\nLINKED ALLOCATION FAILED!\n");
        printf("Reason : Only %d free blocks are available on the disk,\n",
countFreeBlocks());
        printf("but the file needs %d blocks.\n", fileSize);
        return;
    }

    for (i = 0; i < total_blocks && count < fileSize; i++)
    {
        if (disk[i] == 0)
        {
            disk[i] = 1;
            allocated[count] = i;
            count++;
        }
    }

    printf("\nLinked Allocation Successful!\n");
    printf("(Blocks need NOT be contiguous -- each block stores a pointer\n");
    printf("to the next block of the file, just like a linked list.)\n");
    printf("Allocated Block Chain : ");
    for (i = 0; i < count; i++)
    {
        printf("%d", allocated[i]); if (i !=
count - 1) printf(" -> ");
    }
    printf("\n");
/* -----3. Indexed Allocation ----- */

void indexedAllocation(int fileSize)
{

    int indexBlock = -1; int
dataBlocks[MAX]; int
count = 0;

```

```

int i;

/* need fileSize data blocks + 1 index block */
if (countFreeBlocks() < fileSize + 1)
{
    printf("\nINDEXED ALLOCATION FAILED!\n");
    printf("Reason : Need %d blocks (%d data + 1 index) but only %d\n",
        fileSize + 1, fileSize, countFreeBlocks());
    printf("free blocks are available.\n");
    return;
}

/* first free block becomes the index block */
for (i = 0; i < total_blocks; i++)
{
    if (disk[i] == 0)
    {
        indexBlock = i;
        disk[i] = 1;
        break;
    }
}

/* remaining free blocks become the data blocks */
for (i = 0; i < total_blocks && count < fileSize; i++)
{
    if (disk[i] == 0)
    {
        disk[i] = 1;
        dataBlocks[count] = i;
        count++;
    }
}

printf("\nIndexed Allocation Successful!\n");
printf("Index Block %d stores the pointers to every data block.\n", indexBlock);
printf("Index Block Number      : %d\n", indexBlock);
printf("Data Blocks (in index) : ");
for (i = 0; i < count; i++)
    printf("%d ", dataBlocks[i]);
printf("\n");
}

int main()
{
    printf("\n");

```

```

int fileSize, method, i;

printf("=== FILE ALLOCATION METHODS SIMULATION ===\n");
printf("Enter total number of disk blocks: ");
scanf("%d", &total_blocks);

if (total_blocks > MAX)
{
    printf("Disk size exceeds simulator limit of %d blocks.\n", MAX);
    return 0;
}

printf("Enter current status of each block (0-Free , 1-Already Allocated)\n");
printf("(This lets us demonstrate fragmentation on a partly used disk)\n");
for (i = 0; i < total_blocks; i++)
{
    printf("Block %d : ", i);
    scanf("%d", &disk[i]);

```

```

    }

    printf("Enter file size (number of blocks required): ");
    scanf("%d", &fileSize);

    if (fileSize <= 0 || fileSize > total_blocks)
    {
        printf("Invalid file size.\n");
        return 0;
    }

    printf("Choose Allocation Method:\n");
    printf("1. Contiguous\n2. Linked\n3. Indexed\n");
    printf("Enter your choice: ");
    scanf("%d", &method);

    switch (method)
    {
        case 1: contiguousAllocation(fileSize); break;
        case 2: linkedAllocation(fileSize);           break;
        case 3: indexedAllocation(fileSize);          break;
        default: printf("Invalid method choice.\n");
    }

    displayDiskStatus();
    return 0;
}

```

Sample Input & Output

Case 1 - Contiguous allocation FAILS due to fragmentation

Input:

Total disk blocks	: 10	
Initial block status	: 1 0 1 0 1 0 1 0 1 0	(alternate free/allocated -> fragmented)
File size required	: 2 blocks	
Allocation method	: 1 (Contiguous)	

Output:

CONTIGUOUS ALLOCATION FAILED!
Reason : No set of 2 CONTIGUOUS free blocks is available.
This is a classic case of EXTERNAL FRAGMENTATION -- free blocks exist on the disk, but they are scattered and not adjacent to each other, so a contiguous run cannot be formed.

Current Disk Block Status (0-Free, 1-Allocated):

Block 0	: 1	Block 1	: 0	Block 2	: 1	Block 3	: 0	Block 4	: 1
Block 5	: 0	Block 6	: 1	Block 7	: 0	Block 8	: 1	Block 9	: 0

Case 2 - Linked allocation SUCCEEDS on the same fragmented disk

Input:

Total disk blocks	: 10	
Initial block status	: 1 0 1 0 1 0 1 0 1 0	(same fragmented disk as above)

File size required	: 3 blocks
Allocation method	: 2 (Linked)

Output:

Linked Allocation Successful!
 (Blocks need NOT be contiguous -- each block stores a pointer to the next block of the file, just like a linked list.) Allocated Block Chain : 1 -> 3 -> 5 -> NULL

Current Disk Block Status (0-Free, 1-Allocated):
 Block 0:1 Block 1:1 Block 2:1 Block 3:1 Block 4:1
 Block 5:1 Block 6:1 Block 7:0 Block 8:1 Block 9:0

Case 3 - Indexed allocation SUCCEEDS on the same fragmented disk**Input:**

Total disk blocks	: 10	
Initial block status	: 1 0 1 0 1 0 1 0 1 0	(same fragmented disk as above)
File size required	: 3 blocks	
Allocation method	: 3 (Indexed)	

Output:

Indexed Allocation Successful!
 Index Block 1 stores the pointers to every data block.
 Index Block Number : 1
 Data Blocks (in index) : 3 5 7

Current Disk Block Status (0-Free, 1-Allocated): Block 0:1
 Block 1:1 Block 2:1 Block 3:1 Block 4:1 Block 5:1 Block 6:1
 Block 7:1 Block 8:1 Block 9:0

Case 4 - Contiguous allocation SUCCEEDS (enough continuous free space)**Input:**

Total disk blocks	: 10	
Initial block status	: 0 0 0 1 1 0 0 0 1 0	(blocks 0-2 free and contiguous)
File size required	: 3 blocks	
Allocation method	: 1 (Contiguous)	

Output:

Contiguous Allocation Successful!
 File occupies blocks 0 to 2
 Allocated Block List : 0 1 2

Current Disk Block Status (0-Free, 1-Allocated): Block 0:1
 Block 1:1 Block 2:1 Block 3:1 Block 4:1 Block 5:0 Block 6:0
 Block 7:0 Block 8:1 Block 9:0

Question 2: Free Space Management using the Bitmap Technique

[5 Marks]

Question as given: Develop a program to manage free disk space using a bitmap technique. The program should accept inputs including the total number of disk blocks, the initial bitmap (where 0 represents free and 1 represents allocated), and a request for allocation specifying the number of blocks required. The program must identify and allocate free blocks efficiently, update the bitmap after allocation, and also support deallocation of specified blocks with appropriate bitmap updates. The output should display the updated bitmap after allocation and deallocation operations. Ensure that the program efficiently searches for free blocks and properly handles cases where sufficient space is not available.

What This Question Is Asking

- This question asks for the bitmap (bit-vector) method of free-space tracking: one bit per disk block, 0 for free and 1 for allocated.
- The program must be able to (a) allocate N free blocks on request, updating the bitmap, and (b) deallocate specific blocks the user names, also updating the bitmap.
- “Efficiently searches” means the search should avoid checking every single bit blindly when it can skip regions that are already fully allocated — here blocks are examined in groups of 8 and a fully-allocated group is skipped in one comparison.
- The program must also detect and report the case where the free block count is less than what was requested.

Program Logic

- Store the bitmap in `bitmap[MAX]` (0/1 per block).
- Allocation: first count total free blocks; if fewer than requested, fail immediately.
- Otherwise scan the bitmap 8 blocks (one byte-group) at a time; if an entire group is already 1, skip it in a single check; otherwise inspect its bits individually and allocate any 0 found.
- Deallocation: validate the given block numbers are within range, then set their bitmap entry back to 0.
- Print the bitmap as a single string of 0s/1s before allocation, after allocation, and after deallocation.

C Program

```
/* =====
Q2: Free Space Management using the BITMAP (Bit Vector) Technique
```

```

    0 = free block , 1 = allocated block
    - Allocates the requested number of free blocks
    - Updates and displays the bitmap after allocation
    - Deallocates specified blocks and updates the bitmap
    No built-in dynamic memory functions are used (static arrays only).
    ===== */

#include <stdio.h>

#define MAX 100

int bitmap[MAX];
int total_blocks;

void displayBitmap()
{
    int i;
    printf("Bitmap : ");
    for (i = 0; i < total_blocks; i++)
        printf("%d", bitmap[i]);
    printf("\n");
}

/* Efficient free-block search:

    blocks are examined 8 at a time (like scanning a byte of the bitmap). If all 8
    bits in a group are already 1 (fully allocated), the whole group is skipped in a
    single check instead of testing every bit. */
void allocateBlocks(int need) {

    int allocatedList[MAX];
    int count = 0;
    int g, i, groupFull;

    /* quick free-space check first */
    int freeCount = 0;
    for (i = 0; i < total_blocks; i++)
        if (bitmap[i] == 0)
            freeCount++;

    if (freeCount < need)
    {
        printf("\nAllocation FAILED: requested %d blocks but only %d free blocks are
available.\n",
            need, freeCount);
        return;
    }

    for (g = 0; g < total_blocks && count < need; g += 8)
    {
        groupFull = 1;
        for (i = g; i < g + 8 && i < total_blocks; i++)
        {
            if (bitmap[i] == 0)
            {
                groupFull = 0;
                break;
            }
        }
        if (groupFull)
            continue; /* skip this group of 8 blocks entirely */

        for (i = g; i < g + 8 && i < total_blocks && count < need; i++)
        {
            if (bitmap[i] == 0)
            {

```



```

        bitmap[i] = 1;
        allocatedList[count] = i;
        count++;
    }
}

printf("\nAllocation Successful! Blocks allocated : ");
for (i = 0; i < count; i++)
    printf("%d ", allocatedList[i]);
printf("\n");
}
printf("Updated Bitmap After Allocation:\n");
displayBitmap();
void deallocateBlocks(int count, int blocks[]) {

    int i, valid = 1;

    for (i = 0; i < count; i++)
    {
        if (blocks[i] < 0 || blocks[i] >= total_blocks)
        {
            printf("Block %d is out of range (0 - %d). Deallocation aborted.\n",
                blocks[i], total_blocks - 1);
            valid = 0;
        }
    }
    if (!valid)
        return;

    for (i = 0; i < count; i++)
    {
        if (bitmap[blocks[i]] == 0)
            printf("Note: Block %d was already free.\n", blocks[i]);
        bitmap[blocks[i]] = 0;
    }
}
printf("\nDeallocation Successful!\n");
printf("Updated Bitmap After Deallocation:\n");
displayBitmap();
int main()

{
    int i, request, dealloc_count;
    int dealloc_list[MAX];

    printf("=== FREE SPACE MANAGEMENT USING BITMAP ===\n");
    printf("Enter total number of disk blocks: ");
    scanf("%d", &total_blocks);

    if (total_blocks > MAX)
    {
        printf("Disk size exceeds simulator limit of %d blocks.\n", MAX);
        return 0;
    }

    printf("Enter the initial bitmap (0-Free, 1-Allocated), one value per block:\n");
    for (i = 0; i < total_blocks; i++)
    {
        printf("Block %d : ", i);
        scanf("%d", &bitmap[i]);
    }
}

```

```

printf("\nInitial Bitmap:\n");
displayBitmap();

printf("\nEnter number of blocks required for allocation: ");
scanf("%d", &request);
allocateBlocks(request);

printf("\nEnter number of blocks to deallocate: ");
scanf("%d", &dealloc_count);
printf("Enter the block numbers to deallocate:\n");
for (i = 0; i < dealloc_count; i++)
    scanf("%d", &dealloc_list[i]);

deallocBlocks(dealloc_count, dealloc_list);

return 0;
}

```

Sample Input & Output

Sample Run

Input:

```

Total disk blocks           : 16
Initial bitmap              : 0100110001110010
Blocks required (allocate) : 5
Blocks to deallocate (count): 2
Block numbers to deallocate : 0 6

```

Output:

```

Initial Bitmap:
Bitmap : 0100110001110010

Allocation Successful! Blocks allocated : 0 2 3 6 7
Updated Bitmap After Allocation:
Bitmap : 111111101110010

Deallocation Successful!
Updated Bitmap After Deallocation:
Bitmap : 0111110101110010

```

Question 3: Disk Scheduling — FCFS, SSTF and SCAN

[5 Marks]

Question as given: Implement a program that simulates at least three disk scheduling algorithms: FCFS, SSTF, and SCAN. The program should take as input the disk request queue, the initial head position, and the disk size. It must compute the total seek time for each algorithm, display the order in which disk requests are serviced, and compare their performance. The output should include the seek sequence and total seek time for each algorithm, along with identification of the best-performing algorithm based on minimum seek time.

What This Question Is Asking

- The question wants three separate scheduling strategies run on the exact same request queue and head position, so their performance can be compared fairly.
- FCFS simply services requests in the order they arrive.
- SSTF always jumps next to whichever pending request is numerically closest to the current head position.
- SCAN (the “elevator” algorithm) sweeps the head in one direction servicing every request it passes, goes all the way to the end of the disk, then reverses direction for the remaining requests.
- The program must add up head movement (seek time) for each method and declare the one with the smallest total as the winner.

Program Logic

- FCFS: total = sum of $|\text{request}[i] - \text{previous position}|$ taken in input order.
- SSTF: repeatedly pick the not-yet-serviced request with the smallest $|\text{request} - \text{current head}|$, mark it visited, move the head there.
- SCAN: sort the requests; service all requests on the chosen side of the head in order, continue to the disk boundary (0 or $\text{disk_size}-1$), then service the remaining requests on the way back.
- Compare the three totals with simple if/else and print the minimum as the best algorithm.

C Program

```
/* =====
Q3: Disk Scheduling Algorithms - FCFS, SSTF, SCAN
   Computes the seek sequence and total seek time (head movement)
   for each algorithm and reports which one performs best.
   No built-in dynamic memory functions are used (static arrays only).
===== */

#include <stdio.h>
```

```

#include <stdlib.h>

#define MAX 100

int requests[MAX], n;
int head, disk_size;

int absVal(int x) { return x < 0 ? -x : x; }

/* ----- 1. FCFS ----- */
int fcfs(int seq[])
{
    int i, cur = head, total = 0, k = 0;
    for (i = 0; i < n; i++)
    {
        total += absVal(requests[i] - cur);
        cur = requests[i];
        seq[k++] = cur;
    }
    return total;
}

/* ----- 2. SSTF ----- */
int sstf(int seq[])
{
    int visited[MAX] = {0};
    int cur = head, total = 0, k = 0, i;

    for (k = 0; k < n; k++)
    {
        int minDiff = 1000000, minIdx = -1;
        for (i = 0; i < n; i++)
        {
            if (!visited[i] && absVal(requests[i] - cur) < minDiff)
            {
                minDiff = absVal(requests[i] - cur);
                minIdx = i;
            }
        }
        visited[minIdx] = 1;
        total += minDiff;
        cur = requests[minIdx];
        seq[k] = cur;
    }
}

/* ----- 3. SCAN (elevator algorithm) ----- */
/* direction: 1 = moving towards higher block numbers first
0 = moving towards lower block numbers first
returning total */

int scan(int seq[], int direction) {

    int temp[MAX + 2];
    int i, j, cur = head, total = 0, k = 0;

    for (i = 0; i < n; i++)
        temp[i] = requests[i];

    /* simple ascending sort */ for
    (i = 0; i < n - 1; i++)
        for (j = 0; j < n - 1 - i; j++)
            if (temp[j] > temp[j + 1])
            {
                int t = temp[j]; temp[j] = temp[j + 1]; temp[j + 1] = t;
            }

```

```

if (direction == 1) /* head moves towards higher end first */
{
    for (i = 0; i < n; i++) if
        (temp[i] >= cur)
        {
            total += absVal(temp[i] - cur);
            cur = temp[i];
            seq[k++] = cur;
        }
    /* go to the last cylinder (end of disk) */
    total += absVal((disk_size - 1) - cur);
    cur = disk_size - 1;
    seq[k++] = cur;

    for (i = n - 1; i >= 0; i--)
        if (temp[i] < head)
        {
            total += absVal(temp[i] - cur);
            cur = temp[i];
            seq[k++] = cur;
        }
}
else /* direction == 0, head moves towards lower end (0) first */
{
    for (i = n - 1; i >= 0; i--)
        if (temp[i] <= cur)
        {
            total += absVal(cur - temp[i]);
            cur = temp[i];
            seq[k++] = cur;
        }
    /* go to the first cylinder (block 0) */
    total += absVal(cur - 0);
    cur = 0;
    seq[k++] = cur;

    for (i = 0; i < n; i++)
        if (temp[i] > head)
        {
            total += absVal(temp[i] - cur);
            cur = temp[i];
            seq[k++] = cur;
        }
}

void printSeq(int seq[], int k, int total, char *name) {
    return k;

    int i;
    printf("\n%s\n", name);
    printf("Seek Sequence : %d -> ", head);
}
for (i = 0; i < k; i++)
{
    int main()
    {
        printf("%d", seq[i]);
        if (i != k - 1) printf(" -> ");
    }
    printf("\nTotal Seek Time (head movements) : %d\n", total);

    int i, direction;
    int seq1[MAX], seq2[MAX], seq3[MAX + 2];

```

```

int t1, t2, t3, k3;

printf("===DISK SCHEDULINGFCFS, SSTF, SCAN ===\n");
printf("Enter disksize (max cylinder number+ 1): ");
scanf("%d", &disk_size);

printf("Enter initial head position: ");
scanf("%d", &head);

printf("Enter numberof requests in the queue: ");
scanf("%d", &n);

printf("Enter the%d disk requests:\n", n);
for (i = 0; i < n; i++)
    scanf("%d", &requests[i]);

printf("Enter direction for SCAN(1-towards higher end, 0-towards lower end): ");
scanf("%d", &direction);

t1 = fcfs(seq1);
printSeq(seq1,n,t1, "FCFS (First ComeFirst Served)");

t2 = sstf(seq2);
printSeq(seq2,n,t2, "SSTF (Shortest Seek Time First)");

k3 = scan(seq3, direction);
t3 = 0;
/* recomputetotaldistance for SCANfrom returned sequence for display consistency */
{
    int cur = head, s;
    for(s= 0; s<k3; s++) { t3 += absVal(seq3[s] - cur); cur = seq3[s]; }
}
printSeq(seq3,k3,t3, "SCAN (Elevator Algorithm)");

printf("\n--- PERFORMANCOMPARISON ---\n");
printf("FCFS TotalSeek Tim : %d\n", t1);
printf("SSTF TotalSeek e : %d\n", t2);
printf("SCAN TotalSeek Tim : %d\n", t3);
e
if (t1 <= t2 && t1 <= t3) Tim
    printf("Best Performing Algorithm : FCF (minimum total seek time = %d)\n", t1);
else if(t2<=t1&& t2 <= t3)
    printf("Best Performing Algorithm : S (minimum total seek time = %d)\n", t2);
else
    printf("Best Performing Algorithm : SST (minimum total seek time = %d)\n", t3);
F
return0;
}
SCA

```

N

Sample Input & Output

Sample Run

Input:

```

Disk size (cylinders 0-199) : 200
Initial head position       : 50
Number of requests         : 8
Request queue               : 82 170 43 140 24 16 190 25
Direction for SCAN          : 1 (towards higher cylinder numbers)

```

Output:

FCFS (First Come First Served)

Seek Sequence : 50 -> 82 -> 170 -> 43 -> 140 -> 24 -> 16 -> 190 -> 25

Total Seek Time (head movements) : 807

SSTF (Shortest Seek Time First)

Seek Sequence : 50 -> 43 -> 25 -> 24 -> 16 -> 82 -> 140 -> 170 -> 190

Total Seek Time (head movements) : 208

SCAN (Elevator Algorithm)

Seek Sequence : 50 -> 82 -> 140 -> 170 -> 190 -> 199 -> 43 -> 25 -> 24 -> 16 Total Seek Time (head movements) : 332

--- PERFORMANCE COMPARISON ---

FCFS Total Seek Time : 807

SSTF Total Seek Time : 208

SCAN Total Seek Time : 332

Best Performing Algorithm : SSTF (minimum total seek time = 208)

Question 4: Disk Scheduling — C-SCAN and LOOK [5 Marks]

Question as given: Write a program to implement the C-SCAN and LOOK disk scheduling algorithms. The program should accept inputs including the request queue, the initial head position, and the direction of head movement (left or right). It must simulate both algorithms, calculate the total head movement required to service all requests, and compare their efficiency. The output should display the seek sequence, total head movement, and a conclusion indicating which algorithm performs better along with a brief justification. Optionally, the program may include a graphical visualization of head movement.

What This Question Is Asking

- This question focuses on two refinements of SCAN. C-SCAN treats the disk as circular: the head always travels in the same chosen direction, goes all the way to the disk's end, then jumps straight back to the opposite end without servicing on the way, and continues.
- LOOK behaves like SCAN but is smarter about wasted travel — it reverses direction as soon as it has serviced the farthest request in the current direction, instead of travelling all the way to the physical end of the disk.
- The user supplies a direction (left/right) telling the head which way to sweep first.
- The program must total the head movement for both and explain, in words, why one came out ahead.

Program Logic

- Sort the request queue.
- C-SCAN: service requests on the chosen side in order, move to that disk boundary, jump to the opposite boundary (this jump distance is counted as movement), then service the remaining requests in the same direction.
- LOOK: service requests on the chosen side in order only up to the last pending request (no travel to the physical disk end), then reverse and service the rest.
- Sum |difference| between consecutive positions in each sequence to get total head movement, then compare the two totals.

C Program

```
/* =====
Q4: Disk Scheduling Algorithms - C-SCAN and LOOK
  Computes the seek sequence and total head movement for both
  algorithms and compares their efficiency.
  No built-in dynamic memory functions are used (static arrays only).
===== */
```



```

#include <stdio.h>

#define MAX 100

int requests[MAX], n;
int head, disk_size;

int absVal(int x) { return x < 0 ? -x : x; }

void sortArr(int a[], int size)
{
    int i, j, t;
    for (i = 0; i < size - 1; i++)
        for (j = 0; j < size - 1 - i; j++)
            if (a[j] > a[j + 1])
            {
                t = a[j]; a[j] = a[j + 1]; a[j + 1] = t;
            }
}

/* direction : 1 = right (towards higher cylinder numbers)
(towards lower cylinder numbers)

0 = left */

/* ----- C-SCAN : goes to the disk END, then jumps to the OTHER
end and continues servicing in the SAME direction ----- */
int cscan(int seq[], int direction)
{
    int temp[MAX];
    int i, cur = head, k = 0;

    for (i = 0; i < n; i++) temp[i] = requests[i];
    sortArr(temp, n);

    if (direction == 1)
    {
        for (i = 0; i < n; i++)
            if (temp[i] >= cur) seq[k++] = temp[i];

        seq[k++] = disk_size - 1; /* go to the last cylinder */
        seq[k++] = 0; /* jump (circular) to the first block */

        for (i = 0; i < n; i++)
            if (temp[i] < head) seq[k++] = temp[i];
    }
    else
    {
        for (i = n - 1; i >= 0; i--)
            if (temp[i] <= cur) seq[k++] = temp[i];

        seq[k++] = 0; /* go to the first cylinder */
        seq[k++] = disk_size - 1; /* jump (circular) to the last block */

        for (i = n - 1; i >= 0; i--)
            if (temp[i] > head) seq[k++] = temp[i];
    }
    return k;
}

/* ----- LOOK : Like C-SCAN but only goes jump as the head is
in that direction (does not touch disk ends) ----- */
int look(int seq[], int direction) {
    int temp[MAX];
    int i, cur = head, k = 0;

```

```

    for (i = 0; i < n; i++) temp[i] = requests[i];
    sortArr(temp, n);

    if (direction == 1)
    {
        for (i = 0; i < n; i++)
            if (temp[i] >= cur) seq[k++] = temp[i];
        for (i = n - 1; i >= 0; i--)
            if (temp[i] < head) seq[k++] = temp[i];
    }
    else
    {
        for (i = n - 1; i >= 0; i--)
            if (temp[i] <= cur) seq[k++] = temp[i];
        for (i = 0; i < n; i++)
            if (temp[i] > head) seq[k++] = temp[i];
    }
    return k;
}

int totalMovement(int seq[], int k)
{
    int i, cur = head, total = 0;
    for (i = 0; i < k; i++)
    {
        total += absVal(seq[i] - cur);
        cur = seq[i];
    }
}

void printSeq(int seq[], int k, char *name) {
    int i;
    printf("\n%s\n", name);
    printf("Seek Sequence : %d -> ", head);
    for (i = 0; i < k; i++)
    {
        printf("%d", seq[i]);
        if (i != k - 1) printf(" -> ");
    }
    printf("\n");
}

int main()
{
    int i, direction;
    int seqC[MAX + 2], seqL[MAX];
    int kC, kL, totC, totL;
    char dirCh;

    printf("=== DISK SCHEDULING: C-SCAN and LOOK ===\n");
    printf("Enter disk size (max cylinder number + 1): ");
    scanf("%d", &disk_size);

    printf("Enter initial head position: ");
    scanf("%d", &head);

    printf("Enter number of requests in the queue: ");
    scanf("%d", &n);

    printf("Enter the %d disk requests:\n", n);
    for (i = 0; i < n; i++)
        scanf("%d", &requests[i]);

```

```

printf("Enter direction of head movement (r = right/higher, l = left/lower): ");
scanf(" %c", &dirCh);
direction=(dirCh=='r' ? 1 : 0;

kC = cscan(seqC, direction); totC
=totalMovement(seqC, kC);
printSeq(seqC,kC,"C-SCAN (Circular SCAN)");
printf("Total HeadMovement %d\n", totC);

kL = look(seqL, direction);
totL =totalMovement(seqL, kL);
printSeq(seqL,kL,"LOOK");
printf("Total HeadMovement %d\n", totL);

printf("\n--- PERFORMANCE COMPARISON ---\n");
SCANTotal Total Head Movement : %d\n", totC);
printf("LOOK Head Movement : %d\n", totL);

if (totL < totC)
    printf("Conclusion : LOOK performs better. It reverses direction as\n"
           "soon as the last request in the current direction is\n"
           "served, avoiding the wasted trip to the disk boundary\n"
           "that C-SCAN always makes.\n");
else if (totC < totL)
    printf("Conclusion : C-SCAN performs better for this request set.\n"
           "Even though it travels to the disk end, its uniform\n"
           "circular wait time gives more consistent service overall.\n");
else
    printf("Conclusion : Both algorithms give the same total head\n"
           "movement for this particular request set.\n");

return 0;
}

```

Sample Input & Output

Sample Run

Input:

```

Disk size (cylinders 0-199) : 200
Initial head position       : 50
Number of requests         : 8
Request queue              : 82 170 43 140 24 16 190 25
Direction of head movement : r (right / higher cylinder numbers)

```

Output:

```

C-SCAN (Circular SCAN)
Seek Sequence : 50 -> 82 -> 140 -> 170 -> 190 -> 199 -> 0 -> 16 -> 24 -> 25 -> 43
Total Head Movement : 391

LOOK
Seek Sequence : 50 -> 82 -> 140 -> 170 -> 190 -> 43 -> 25 -> 24 -> 16
Total Head Movement : 314

--- PERFORMANCE COMPARISON ---
C-SCAN Total Head Movement : 391
LOOK Total Head Movement : 314
Conclusion : LOOK performs better. It reverses direction as
soon as the last request in the current direction is

```

serviced, avoiding the wasted trip to the disk boundary that C-SCAN always makes.

Question 5: Integrating File Allocation Strategies with Disk Scheduling

[5 Marks]

Question as given: Design a program that integrates file allocation strategies with disk scheduling techniques to optimize system performance. The program should accept inputs such as the total number of disk blocks, file size (number of blocks required), disk request queue, initial head position, allocation method (Contiguous or Indexed), and disk scheduling method (FCFS or SSTF). It must allocate disk blocks based on the chosen allocation strategy, simulate disk scheduling to access the allocated blocks, calculate the total seek time, and compare performance across different combinations. The output should display allocated blocks, seek sequence, total seek time, and identify the best combination of allocation and scheduling methods. Ensure the program handles allocation failures, avoids duplicate block allocation, and maintains efficient scheduling.

What This Question Is Asking

- This is a combined question: first a file must actually be allocated space on the disk (using Contiguous or Indexed allocation), and then the disk arm must be scheduled to visit the blocks that were allocated (using FCFS or SSTF), together with any other pending disk requests.
- “Compare performance across different combinations” means the program should not just run one allocation + one scheduling method, but try Contiguous and Indexed allocation, and for each one that succeeds, run both FCFS and SSTF, so all valid (allocation, scheduling) pairs can be compared side by side.
- “Handles allocation failures” means that if, say, Contiguous allocation cannot find a free run, the program should skip that combination gracefully and continue with the ones that are possible, rather than crashing.
- “Avoids duplicate block allocation” means the same block should not be counted twice when the freshly-allocated blocks are merged with the externally pending disk request queue before scheduling.

Program Logic

- Try Contiguous allocation on one copy of the disk map, and Indexed allocation on a separate copy of the disk map, so one method's success/failure never affects the other.
- For each allocation method that succeeds, merge its allocated block numbers with the externally pending disk requests, removing duplicates, to build the disk-scheduling request queue.

- Run FCFS and SSTF on that merged queue for every allocation method that succeeded (up to 4 combinations: Contiguous+FCFS, Contiguous+SSTF, Indexed+FCFS, Indexed+SSTF).
- Track the minimum total seek time seen across all valid combinations and report it as the best (allocation method + scheduling method) pair.

C Program

```

/* =====
Q5: Integration of File Allocation Strategies with Disk Scheduling
- Allocates the requested file using Contiguous and Indexed
  allocation (so failures in one method can still be compared
  with the other).
- Merges the allocated blocks with any pending external disk
  requests (duplicates removed).
- Runs FCFS and SSTF scheduling on the merged queue for every
  allocation method that succeeded.
- Reports the total seek time of every valid combination and
  identifies the overall best (Allocation + Scheduling) pair.
No built-in dynamic memory functions are used (static arrays only).
===== */

#include <stdio.h>

#define MAX 100

int disk[MAX];          /* master free/allocated map of the disk      */
int total_blocks;
int head, extReq[MAX], extCount;

int absVal(int x) { return x < 0 ? -x : x; }

int countFree(int d[])
{
    int i, c = 0;
    for (i = 0; i < total_blocks; i++) if (d[i] == 0) c++;
    return c;
}

/* ---- Allocation strategies (each works on its OWN copy of the disk) ---- */ /* returns 1 on
success and fills 'result' with the allocated blocks,

'count' with how many. returns 0 on failure. */
int contiguousAlloc(int d[], int fileSize, int result[], int *count) {

    int i, j, start = -1, found;
    for (i = 0; i <= total_blocks - fileSize; i++)
    {
        found = 1;
        for (j = i; j < i + fileSize; j++)
            if (d[j] == 1) { found = 0; break; }
        if (found) { start = i; break; }
    }
    if (start == -1) { *count = 0; return 0; }

    *count = 0;
    for (j = start; j < start + fileSize; j++)
    {
        d[j] = 1;
        result[(*count)++] = j;
    }
}

```

```

    return 1;
}

int indexedAlloc(int d[], int fileSize, int result[], int *count) {

    int i, idx = -1, n = 0;
    if (countFree(d) < fileSize + 1) { *count = 0; return 0; }

    for (i = 0; i < total_blocks; i++)
        if (d[i] == 0) { idx = i; d[i] = 1; break; }

    result[n++] = idx; /* result[0] = index block, rest = data blocks */
    for (i = 0; i < total_blocks && n < fileSize + 1; i++)
        if (d[i] == 0) { d[i] = 1; result[n++] = i; }
    /* ---- Build the scheduling queue: allocated blocks + external

    *count = n;
    return 1;

    requests, with duplicates removed ----- */
int buildQueue(int allocBlocks[], int allocCount, int q[])
{
    int i, j, k = 0, dup;

    for (i = 0; i < allocCount; i++)
    {
        dup = 0;
        for (j = 0; j < k; j++) if (q[j] == allocBlocks[i]) dup = 1;
        if (!dup) q[k++] = allocBlocks[i];
    }
    for (i = 0; i < extCount; i++)
    {
        dup = 0;
        for (j = 0; j < k; j++) if (q[j] == extReq[i]) dup = 1;
        if (!dup) q[k++] = extReq[i];
    }
    return k;
}

/* ---- Scheduling algorithms ---- */ int
fcfsTime(int q[], int n)
{
    int i, cur = head, total = 0;
    for (i = 0; i < n; i++) { total += absVal(q[i] - cur); cur = q[i]; }
    return total;
}

int sstfTime(int q[], int n)
{
    int visited[MAX] = {0};
    int cur = head, total = 0, i, k;
    for (k = 0; k < n; k++)
    {
        int minDiff = 1000000, minIdx = -1;
        for (i = 0; i < n; i++)
            if (!visited[i] && absVal(q[i] - cur) < minDiff)
                { minDiff = absVal(q[i] - cur); minIdx = i; }
        visited[minIdx] = 1;
        total += minDiff;
        cur = q[minIdx];
    }
    return total;
}

```

```

int main() {

    int fileSize, i;
    int diskCopy1[MAX], diskCopy2[MAX];
    int contigBlocks[MAX], contigCount, contigOK;
    int idxBlocks[MAX], idxCount, idxOK;
    int q[MAX], qn;
    int bestTime = 1000000; char bestCombo[60] = "";

    printf("=== INTEGRATED FILE ALLOCATION + DISK SCHEDULING ===\n");
    printf("Enter total number of disk blocks: ");
    scanf("%d", &total_blocks);

    printf("Enter current status of each block (0-Free, 1-Allocated):\n");
    for (i = 0; i < total_blocks; i++)
    {
        printf("Block %d : ", i);
        scanf("%d", &disk[i]);
    }

    printf("Enter file size (number of blocks required): ");
    scanf("%d", &fileSize);

    printf("Enter number of PENDING external disk requests: ");
    scanf("%d", &extCount);
    printf("Enter the %d external disk requests:\n", extCount);
    for (i = 0; i < extCount; i++)
        scanf("%d", &extReq[i]);

    printf("Enter initial disk head position: ");
    scanf("%d", &head);

    /* ---- Try both allocation strategies on independent disk copies ---- */
    for (i = 0; i < total_blocks; i++) diskCopy1[i] = disk[i];
    for (i = 0; i < total_blocks; i++) diskCopy2[i] = disk[i];

    contigOK = contiguousAlloc(diskCopy1, fileSize, contigBlocks, &contigCount);
    idxOK     = indexedAlloc(diskCopy2, fileSize, idxBlocks, &idxCount);

    printf("\n--- ALLOCATION PHASE ---\n");
    if (contigOK)
    {
        printf("Contiguous Allocation SUCCESS -> Blocks: ");
        for (i = 0; i < contigCount; i++) printf("%d ", contigBlocks[i]);
        printf("\n");
    }
    else
        printf("Contiguous Allocation FAILED (no free contiguous run of %d blocks).\n",
fileSize);

    if (idxOK)
    {
        printf("Indexed Allocation SUCCESS -> Index Block: %d , Data Blocks: ", idxBlocks[0]);
        for (i = 1; i < idxCount; i++) printf("%d ", idxBlocks[i]);
        printf("\n");
    }
    else
        printf("Indexed Allocation FAILED (need %d free blocks, insufficient free space).\n",
fileSize + 1);

    printf("\n--- SCHEDULING PHASE (allocated blocks + pending requests) ---\n");

    if (contigOK)
    {
        qn = buildQueue(contigBlocks, contigCount, q);

```



```

        int t = fcfsTime(q, qn);
        printf("\n[Contiguous + FCFS] Request Queue : ");
        for(i= 0; i<qn; i++) printf("%d ", q[i]);
        printf("\nTotalSeek Time : %d\n", t);
        if(t<bestTime)      { bestTime = t;  sprintf(bestCombo, "Contiguous + FCFS"); }

        qn= buildQueue(contigBlocks, contigCount, q);
        t = sstfTime(q, qn);
        printf("\n[Contiguous + SSTF] Request Queue : ");
        for(i= 0; i<qn; i++) printf("%d ", q[i]);
        printf("\nTotalSeek Time : %d\n", t);
        if(t<bestTime)      { bestTime = t;  sprintf(bestCombo, "Contiguous + SSTF"); }
    }

    if (idxOK)
    {
        qn= buildQueue(idxBLOCKS, idxCount, q);
        int t = fcfsTime(q, qn);
        printf("\n[Indexed + FCFS] Request Queue : ");
        for(i= 0; i<qn; i++) printf("%d ", q[i]);
        printf("\nTotalSeek Time : %d\n", t);
        if(t<bestTime)      { bestTime = t;  sprintf(bestCombo, "Indexed + FCFS"); }

        qn= buildQueue(idxBLOCKS, idxCount, q);
        t = sstfTime(q, qn);
        printf("\n[Indexed + SSTF] Request Queue : ");
        for(i= 0; i<qn; i++) printf("%d ", q[i]);
        printf("\nTotalSeek Time : %d\n", t);
        if(t<bestTime)      { bestTime = t;  sprintf(bestCombo, "Indexed + SSTF"); }
    }

    printf("\n--- FINALCOMPARISON ---\n");
    if (!contigOK && !idxOK)
        printf("Both allocation methodsfailed. No scheduling could be performed.\n");
    else
        printf("Best Combination: %s (Total Seek Time = %d)\n", bestCombo, bestTime);

    return 0;
}

```

Sample Input & Output

Sample Run

Input:

```

Total disk blocks           : 20
Initial block status       : 1 0 1 0 1 0 1 0 1 0 0 0 0 1 1 0 1 0 1 0
File size required         : 4 blocks
Pending external disk requests(n) : 3
External disk requests     : 55 12 18
Initial disk head position : 10

```

Output:

```

--- ALLOCATION PHASE ---
Contiguous Allocation SUCCESS -> Blocks: 9 10 11 12
Indexed Allocation SUCCESS -> Index Block: 1 , Data Blocks: 3 5 7 9

--- SCHEDULING PHASE (allocated blocks + pending requests) ---

```

[Contiguous + FCFS] Request Queue : 9 10 11 12 55 18
Total Seek Time : 84

[Contiguous + SSTF] Request Queue : 9 10 11 12 55 18
Total Seek Time : 47

[Indexed + FCFS] Request Queue : 1 3 5 7 9 55 12 18
Total Seek Time : 112

[Indexed + SSTF] Request Queue : 1 3 5 7 9 55 12 18
Total Seek Time : 63

--- FINAL COMPARISON ---

Best Combination : Contiguous + SSTF (Total Seek Time = 47)

(Note: external request 12 coincided with an allocated block and was not duplicated in the Contiguous queue -- duplicate block requests are automatically removed before scheduling.)